INCLUSÃO SOCIAL ATRAVÉS DO SORRISO

CHALLENGE
1 TDS AGOSTO

tdb

INCLUSÃO SOCIAL ATRAVÉS DO SORRISO

Entregáveis

Calendário

23/03 - Mentoria com TDB na FIAP

10/04 - Sprint 3

18/05 - Banca com professores na FIAP

21/05 - Sprint 4

15/06 - Banca Final de Aprovação para o NEXT na TDB

24/10 - NEXT

Sprint 3

ARTIFICIAL INTELLIGENCE e CHATBOT

SPRINT 03 – Preparo para implementação de modelos preditivos com Machine Learning

As duas primeiras sprints trataram dos aspectos de implementação de um chatbot na solução do seu grupo. As próximas duas sprints terão o foco no aspecto de **inteligência artificial** por meio da implementação de um **modelo preditivo** utilizando *machine learning* (ML).

Nessa sprint 3, o grupo deverá identificar claramente um problema que possa ser enriquecido pelo uso de um componente preditivo utilizando ML, e então encontrar (ou gerar) uma base de dados adequada. Em seguida, deve ser feita a análise exploratória de dados (EDA) sobre a base. A sprint 4, na sequência, será dedicada ao treinamento, validação, documentação e disponibilização do modelo em um servidor acessível via API.

O modelo será consumido posteriormente em uma página web. Portanto, tenham em mente que a base deve ter relação direta com a ideia do projeto que estão propondo solucionar, e deve se relacionar também com entregáveis de outras disciplinas. Em aula discutiremos por que modelos de ML são considerados *modelos derivados de dados,* reforçando a importância de uma análise cuidadosa na sprint 3.

ARTIFICIAL INTELLIGENCE e CHATBOT

Objetivos

- Identificar um problema preditivo relacionado ao tema do projeto do grupo;
- Selecionar ou gerar uma base de dados com potencial para treinar um modelo de ML coerente;
- Realizar uma análise exploratória utilizando gráficos e estatísticas descritivas que apoiem o entendimento de padrões, relações, e limitações dos dados.

Entregáveis e pontuação

- (50 pontos): **Análise exploratória (.ipynb).** Gere no mínimo 5 gráficos, apresentando informações úteis. Ao apresentar cada um dos gráficos, escreva um comentário indicando qual dos itens abaixo podem ser observados como informação útil:
  - Padrões ou tendências temporais;
  - Correlações entre variáveis;
  - Distribuições relevantes para o problema;
  - Estruturas, grupos, ou outliers;
  - Comparações entre grupos.

Gráficos puramente ilustrativos, sem interpretação possível, ou sem relevância, não contarão como válidos.

ARTIFICIAL INTELLIGENCE e CHATBOT

- (30 pontos): **Documentação explicando a base de dados (.pdf)**. Gere um documento que contenha:
  - Breve descrição da base de dados. Em caso de uma base encontrada na internet, indique fonte e justificativa da escolha. Em caso de dados coletados pelos alunos, explique o processo de coleta de dados, a escolha das colunas, e a metodologia utilizada. Em caso de dados gerados, coloque o prompt completo, o racional por trás do prompt, e o que se esperava como resultado. O uso de ferramentas de IA generativa é permitida, porém o prompt deverá ser incluído na descrição, e a pontuação passará a ser relacionada à qualidade do prompt.
  - Identificação da variável alvo. Indique qual das colunas da base pode ser usada como variável alvo (o que o modelo deve aprender a prever). Explicar como essa variável se relaciona ao problema de aplicação.
- (20 pontos): **Arquivo com os dados (.csv)**. Os dados devem ser enviados para avaliação. A base deve estar limpa, sem problemas identificáveis, e deve ser o suficiente para treinar o modelo na sprint 4.

BUILDING RELATIONAL DATABASE

• Criar a implementação prática da estrutura de banco dados utilizando a linguagem Sql (create table) plataforma Oracle.

• Entregar a seguinte documentação: Modelo descritivo, diagrama de classes (BackEnd) que demonstra os objetos criados e usados para armazenamento de dados, estes objetos também devem aparecer na modelagem do projeto: DER ou MER, MLR e MF (Uso obrigatório da ferramenta Data Modeler). **Até 40 pontos**, desde que atenda o solicitado e dentro da metodologia aplicada em aula pelo professor.

• Desenvolver a estrutura usando os comandos DDL da linguagem Sql na plataforma Oracle, lembrando que as tabelas devem corresponder aos objetos criados em BackEnd. Na criação da estrutura deverão aparecer as regras de validação de dados como: Pk, Fk, Nn, Uk e Ck. **Até 60 pontos**, desde que objetos e tabelas sejam correspondentes a documentação e diagrama de classes apresentado no item anterior.

•Formato de entrega, documento em pdf com a documentação solicitada anterirormente, arquivo sql ou txt com a implementação SQL ORACLE.

BUILDING RELATIONAL DATABASE

- Atenção as penalidades:

- Entregar arquivo fora do padrão especificado na documentação > -25% da nota total
- Não usar a linguagem SQL solicitada na documentação > - 50% da nota total
- Usar ferramenta de modelagem diferente da solicitada na documentação > - 25% da nota total
- Não idenficar os integrantes do grupo nos arquivos > - 30% da nota total
- Não criar a codificação DDL inicial para dropar as tabelas > - 50% da nota total
- Instrução com erro de execução será desconsiderada, nota = 0 (zero)

Padrão a ser seguido no arquivo de codificação:

- Primeiras linhas > ra e nome completo em ordem alfabética
- Comentar o que a codificação a seguir irá realizar, por exemplo:
    - Apagando as tabelas....
    - Criando as tabelas.....

COMPUTATIONAL THINKING USING PYTHON

Objetivo:
A equipe deve dar continuida ao projeto apresentado na Sprint 2. O sistema deve contemplar os requisitos descritos a seguir.

Entrega 1 (70 pontos) - Código Fonte (arquivos .py):

- O Código Fonte deve contemplar os seguintes requisitos:
  - [15 pontos] Implementar um CRUD, com pelo menos uma operação de inserção, alteração, exclusão e consulta;
  - [5 pontos] Estrutura de menus para acesso às opções do CRUD;
  - [5 pontos] Utilizar listas e dicionários para armazenar e manipular dados;
  - [10 pontos] Realizar validações em entrada de usuário e tratar exceções utilizando (utilizar try/except);
  - [15 pontos] Organizar o código fonte em funções com passagem de parâmetro e retorno;
  - [15 pontos] Utilizar arquivos de texto ou arquivos json para reaalizar a persistência de dados;
  - [5 pontos] Garantir a usabilidade e a boa experiência do usuário e aplicar boas práticas de programação.

COMPUTATIONAL THINKING USING PYTHON

Entrega 2 (30 pontos) - Vídeo Explicativo:
Produzir um vídeo explicando como o programa aborda o problema e demonstrando o software em funcionamento, com duração máxima de 5 minutos.

- Destacar as funcionalidades implementadas e estruturas de programação utilizadas;
- O vídeo deve ser disponibilizado no Youtube (não esquecer de deixar o vídeo acessível).

Formato de Entrega: Um único arquivo em formato .ZIP contendo:

- Código fonte completo do sistema;
- Arquivo TXT com Nomes e RMs dos alunos do grupo, e o link para acesso ao vídeo disponibilizado no Youtube.

Penalidades:

- [70 pontos] - Soluções que não se adequam ao problema proposto (Não façam implementações genéricas como cadastro de usuários. Escolham funcionalidades específicas do projeto);
- [20 pontos] - CRUD incompleto;
- [30 pontos] - Vídeo não acessível ou link quebrado;
- [30 pontos] - Não persistir dados em arquivos;
- [20 pontos] - Entrega fora dos padrões solicitados.

DOMAIN DRIVEN DESIGN USING JAVA (1 de 7)

Descrição:

Esta sprint do projeto Java tem como objetivo avaliar o desenvolvimento técnico e documental da solução proposta pela equipe. A entrega deve conter tanto o projeto Java funcional quanto a documentação detalhada, permitindo ao avaliador compreender claramente o funcionamento, o escopo e a lógica implementada. A entrega é individual por equipe, e deve ser feita no portal do aluno, em formato .ZIP, contendo a documentação em PDF e a pasta do projeto Java completo.

Requisitos

Documentação (em PDF) - 20 pontos

A documentação deve estar clara, organizada e conter obrigatoriamente os seguintes itens:

Capa [0,25]

Nome dos integrantes.
Nome da equipe.
Nome da solução (sistema desenvolvido).

DOMAIN DRIVEN DESIGN USING JAVA (2 de 7)

Sumário [0,25]

Gerado automaticamente ou manualmente, listando os tópicos da documentação com numeração de páginas.

Objetivo e escopo do projeto [2,50]

Texto claro, conciso e objetivo explicando a solução desenvolvida.
Deve conter o propósito principal do sistema e os limites de sua atuação.

Descrição das funcionalidades [3,50]

Breve explicação das principais funcionalidades implementadas.
Pode ser em forma de lista ou tópicos com descrição.
Explicação de cada método lógico, com o print de cada um dos 4 métodos exigidos.

Protótipo do sistema [4,50]

Telas do sistema (do front-end, mesmo que não estejam implementadas ainda).
Pode ser feito no Figma, PowerPoint, Canva, ou similar.
Cada tela deve vir acompanhada de explicações sobre como o usuário irá interagir com ela.

DOMAIN DRIVEN DESIGN USING JAVA (3 de 7)

Modelo de Entidade-Relacionamento (MER) [3,50]

Diagrama do banco de dados com entidades, atributos, relacionamentos, e chaves.

Diagrama de Classes [3,50]

Atualizado conforme a estrutura da aplicação.
Deve representar as classes, atributos, métodos, relacionamentos (herança, composição etc.).

Procedimentos para rodar a aplicação [2,00]

Lista de instruções claras para executar o projeto.
Deve conter:

Ferramentas utilizadas (ex: Eclipse, NetBeans, IntelliJ, VS Code).
Versões necessárias (Java JDK, banco de dados).
Passo a passo da execução.
Caso necessário, como importar o projeto.

DOMAIN DRIVEN DESIGN USING JAVA (4 de 7)

Projeto Java (Código-Fonte) - 80 Pontos

O código deve estar funcional e estruturado com base nos padrões trabalhados em sala de aula. Os seguintes requisitos devem ser atendidos:

Camada Modelo (entities, beans) [20 pontos]

Classes modelo corretamente estruturadas, com atributos, construtores, métodos getters e setters.
Conter no mínimo 6 classes, padrões de encapsulamento e boa prática.

Métodos com lógica de negócio [20 pontos]

Implementação de no mínimo 4 métodos relevantes.
Os métodos devem conter lógica e regras de negócio condizentes com a proposta do sistema.
A complexidade e a criatividade da solução serão consideradas na avaliação.

DOMAIN DRIVEN DESIGN USING JAVA (5 de 7)

Classe de Conexão com o Banco de Dados [10 pontos]

Deve conter a lógica de conexão com o banco.
Os dados de conexão (usuário e senha) devem estar inseridos no código.

Camada DAO e CRUD completo [20 pontos]

Implementação completa da camada DAO.
Métodos de Create, Read, Update e Delete devem estar funcionais.
As operações devem ser testadas e aplicadas às classes modelo.

Classes de Teste [10 pontos]

Classes com o método main apenas para testar as implementações em sua totalidade.
Retornar também os resultados dos métodos da lógica de negócios definidas para o projeto.
Deve instanciar objetos e simular a utilização da aplicação.

DOMAIN DRIVEN DESIGN USING JAVA (6 de 7)

Penalidades:

Situação	Penalidade
Documentação desorganizada.	- 5 até 20 pontos.
Documentação em outro formato que não seja PDF.	-10 até 20 pontos.
Erro de nomenclatura no projeto.	-10 a -30 pontos (dependendo da gravidade).
Falta de organização no código.	-5 a -20 pontos.
Baixa resolução das imagens na documentação.	-5 pontos por item.
Entrega após o prazo (não serão aceitos)	Penalização total (-100 pontos)

DOMAIN DRIVEN DESIGN USING JAVA (7 de 7)

Penalidades:

Situação	Penalidade
Ausência de qualquer item obrigatório na documentação.	- até 5 pontos por item, além da pontuação atribuída.
Entrega fora do padrão solicitado.	-10 pontos
Código que não compila ou não executa corretamente.	-20 a -50 pontos (dependendo da gravidade)
Falta de pelo menos um dos métodos com lógica de negócio.	-5 a -10 pontos por método faltante
CRUD incompleto ou com falhas graves.	-10 pontos por item
Entrega após o prazo (não serão aceitos).	Penalização total (-100 pontos)

FRONT-END DESIGN ENGINEERING

(01/22)

SPRINT 03 – TRANSIÇÃO PARA COMPONENTIZAÇÃO E SPA COM REACT - VITEJS

Para esta etapa do projeto, as páginas desenvolvidas anteriormente servirão como base para a evolução da solução, agora com foco em **componentização e desenvolvimento moderno de interfaces**.

A aplicação será reestruturada utilizando **REACT + VITE + TYPESCRIPT**, permitindo organização modular, reutilização de componentes e a implementação do conceito de **Single Page Application (SPA)**. Essa abordagem trará maior desempenho, escalabilidade e eficiência ao desenvolvimento.

O uso do **GITHUB como ferramenta de versionamento** continua obrigatório para garantir histórico de desenvolvimento, controle de alterações e colaboração entre os integrantes da equipe.

Tecnologias obrigatórias nesta etapa: REACT, VITE e **TYPESCRIPT**.

FRONT-END DESIGN ENGINEERING

(02/22)

A **SPRINT 03** representa a evolução técnica e estrutural do projeto, migrando do modelo tradicional de páginas HTML para uma aplicação moderna baseada em componentes.

OBJETIVOS DESTA ETAPA

- **Migrar o projeto para React + Vite + TypeScript**

  Transformar o site em uma aplicação moderna, modular e profissional utilizando React com suporte a SPA (Single Page Application).

- **Reaproveitar o conteúdo das sprints anteriores**

  Migrar todas as páginas já desenvolvidas (Home, Integrantes, Sobre, FAQ, Contato e Solução do Projeto) para componentes React, mantendo o layout e identidade visual.

- **Implementar arquitetura de componentes e rotas**

  Organizar a aplicação com componentes reutilizáveis (Header, Footer, Menu, Botões, Cards) e navegação interna com React Router.

- **Garantir responsividade e boa experiência do usuário (UX)**

  Manter o site totalmente responsivo para mobile, tablet e desktop, com navegação fluida e visual consistente.

- **Garantir organização e versionamento profissional**

  Utilizar boas práticas de código com TypeScript, manter estrutura organizada e versionamento contínuo no GitHub.

FRONT-END DESIGN ENGINEERING

ALGUMAS REGRAS (01/05)

- **REESTRUTURAÇÃO TOTAL DO PROJETO EM REACT (Obrigatório)**
  - Todas as páginas já desenvolvidas nas Sprints anteriores **devem ser migradas para React**.
  - A aplicação deve ser uma **SPA** (**Single Page Application**) utilizando **React Router** para navegação.
  - O conteúdo deve ser convertido em **componentes reutilizáveis** com boa organização de pastas.

- **DEFINIÇÃO DAS PÁGINAS DO PROJETO**
  - Todas as páginas obrigatórias da Sprint 02 devem ser mantidas na Sprint 03:
    - Home (index); Integrantes; Sobre; FAQ; Contato ePáginas da Solução do Projeto
  - Além disso:
    - Cada página deverá ser **convertida para um componente React** dentro da pasta /src/pages.
    - **Não será aceita a entrega de páginas faltando.** A ausência de qualquer página obrigatória **resultará em penalidade**.

FRONT-END DESIGN ENGINEERING

ALGUMAS REGRAS (02/05)

- **TECNOLOGIAS OBRIGATÓRIAS**
  - **REACT**

    Interface e Componentização
  - **VITE**

    Build e performance
  - **TYPESCRIPT**

    Tipagem obrigatória no código
  - **TAILWINDCSS**

    Estilização de toda a interface
  - **REACT ROUTER DOM**

    Navegação entre páginas
  - **HOOK FORM (REACT HOOK FORM)**

    Para validação de formulários
  - **GITHUB**

    Versionamento obrigatório

FRONT-END DESIGN ENGINEERING

(05/22)

ALGUMAS REGRAS (03/05)

- **RESPONSIVIDADE (Obrigatório)**
  - A responsividade deve ser mantida com **Tailwind**.
  - O site deve funcionar em: *Mobile* (até 480px), *Tablet* (768px) e *Desktop* (992px+)
  - *Layout* não pode quebrar em nenhuma tela.

- **INTEGRAÇÕES E FORMULÁRIOS**
  - Não haverá consumo de **API** nesta SPRINT 03.
  - Se houver formulários ou entrada de dados, devem obrigatoriamente utilizar **React Hook Form** com validações.
  - **Typescript** deve ser aplicado também nos formulários.

FRONT-END DESIGN ENGINEERING

(06/22)

ALGUMAS REGRAS (04/05)

- **VERSIONAMENTO E HISTÓRICO DE COMMITS**
  - O projeto deve estar versionado no GITHUB.
  - Fluxo GIT obrigatório: *branches* + *commits* regulares.
  - Mínimo: 5 *commits* por integrante e 15 *commits* no total do grupo.
  - Todos os *commits* devem ser significativos e com mensagem clara.
  - O *link* do repositório deve estar no README.

- **PROIBIÇÕES**
  - 🚫 Usar bibliotecas externas de UI (Bootstrap, Material UI, Chakra UI etc.)
  - 🚫 Usar *Axios* ou bibliotecas de requisição HTTP
  - 🚫 Usar *templates* prontos da *internet*
  - 🚫 Copiar códigos já entregues por outros grupos
  - 🚫 Usar *node_modules* no envio do .zip

FRONT-END DESIGN ENGINEERING

ALGUMAS REGRAS (05/05)

- **ENTREGA DO PROJETO**
  - Entregar arquivo .zip com:
    - Projeto React completo
    - Sem a pasta *node_modules*
    - Histórico do GIT incluido
    - *Branch* principal *main*
  - Enviar *link* do repositório GITHUB no README.
- **README OBRIGATÓRIO**
  - O README.md deve conter:
    - Descrição do projeto
    - Tecnologias utilizadas
    - Estrutura de pastas
    - Como executar localmente
    - Link do GitHub
    - Integrantes do grupo

FRONT-END DESIGN ENGINEERING

CRITÉRIOS DE AVALIAÇÃO E PONTUAÇÃO (01/09)

1. CONSTRUÇÃO DA APLICAÇÃO COM REACT + VITEJS + TYPESCRIPT

Avalia a conversão do projeto da SPRINT 02 para REACT + VITEJS + TYPESCRIPT com arquitetura moderna e organização correta.

A. CONVERSÃO PARA FRONT-END MODERNO Este bloco avalia a migração do projeto para React + ViteJS + TypeScript, garantindo páginas convertidas em componentes, navegação SPA funcional, organização profissional do código e uso correto de tipagem.	**PONTUAÇÃO MÁXIMA** **20,0 pontos**
REQUISITO	**PONTUAÇÃO**
a. Conversão das páginas obrigatórias para REACT Todas as páginas (Home, Sobre, FAQ, Contato, Integrantes e Solução) convertidas para componentes React)	5,0 pontos
b. SPA e navegação com React Router Navegação funcional entre páginas usando react-router-dom.	5,0 pontos
c. Estruturação correta do projeto Organização em /src/components e /src/pages, padrão profissional.	5,0 pontos
d. Uso de Typescript Tipagem aplicada corretamente aos componentes e props.	5,0 pontos

FRONT-END DESIGN ENGINEERING

CRITÉRIOS DE AVALIAÇÃO E PONTUAÇÃO (02/09)

2. COMPONENTIZAÇÃO, MODULARIDADE E REUTILIZAÇÃO

Avalia a qualidade da construção dos componentes com React.

A.	**QUALIDADE E ORGANIZAÇÃO DOS COMPONENTES** Avalia a criação de componentes reutilizáveis e bem organizados em React, com código modular e padronizado.	**PONTUAÇÃO MÁXIMA** **15,0 pontos**
	REQUISITO	**PONTUAÇÃO**
a.	**COMPONENTES REUTILIZÁVEIS** Header, Footer, Layout, Cards, Botões etc reaproveitados no código.	8,0 pontos
b.	**MODULARIDADE DA APLICAÇÃO** Divisão lógica do projeto em pastas e arquivos.	4,0 pontos
c.	**ORGANIZAÇÃO DO CÓDIGO E PADRÕES REACT** Nomeação de componentes, pastas, função limpa e clara.	3,0 pontos

FRONT-END DESIGN ENGINEERING

CRITÉRIOS DE AVALIAÇÃO E PONTUAÇÃO (03/09)

3. HOOKS, PROPS E NAVEGAÇÃO DE DADOS

Avalia domínio de React com dados e comportamentos dinâmicos.

A.	**QUALIDADE E ORGANIZAÇÃO DOS COMPONENTES** Avalia a criação de componentes reutilizáveis e bem organizados em React, com código modular e padronizado..	**PONTUAÇÃO MÁXIMA** **20,0 pontos**
	REQUISITO	**PONTUAÇÃO**
a.	**USO DE USESTATE** Estado funcional em pelo menos 2 componentes.	2,0 pontos
b.	**USO DE USEEFFECT** Controle de efeitos/renderização.	4,0 pontos
c.	**USO DE USENAVIGATE E USEPARAMS** Navegação dinâmica entre rotas.	4,0 pontos
d.	**USO CORRETO DE PROPS** Passagem de dados entre componentes	1,0 ponto
e.	**ROTAS ESTÁTICAS** Criação de rotas estáticas	4,0 pontos
f.	**ROTAS DINÂMICAS** Criação de rotas dinâmicas e passagem de parâmetros	5,0 pontos

FRONT-END DESIGN ENGINEERING

CRITÉRIOS DE AVALIAÇÃO E PONTUAÇÃO (04/09)

4. ESTILIZAÇÃO E RESPONSIVIDADE COM TAILWIND

Esse bloco avalia a qualidade visual do projeto e como ele se adapta a diferentes tamanhos de tela utilizando **TailwindCSS**, garantindo um design profissional, organizado e totalmente responsivo para mobile, tablet e desktop.

A.	**ESTILIZAÇÃO E RESPONSIVIDADE COM TAILWIND** Este item avalia a qualidade visual da aplicação, verificando se o layout foi bem estilizado com Tailwind CSS e se a interface funciona corretamente em diferentes tamanhos de tela (responsividade).	**PONTUAÇÃO MÁXIMA** **20,0 pontos**
	REQUISITO	**PONTUAÇÃO**
a.	**LAYOUT PROFISSIONAL** Estilização e Responsividade com TailwindCSS.	5,0 pontos
b.	**TAILWIND APLICADO CORRETAMENTE** Classes bem utilizadas, sem CSS externo.	5,0 pontos
c.	**RESPONSIVIDADE MOBILE** Ajuste para celulares.	3,0 pontos
d.	**RESPONSIVIDADE TABLETS** Ajuste para telas médias	3,0 pontos
e.	**RESPONSIVIDADE DESKTOP** Layout organizado em alta resolução	4,0 pontos

FRONT-END DESIGN ENGINEERING

CRITÉRIOS DE AVALIAÇÃO E PONTUAÇÃO (05/09)

5. FORMULÁRIOS E HOOK FORMS

Esse bloco avalia se os formulários do projeto foram implementados corretamente utilizando React Hook Form, garantindo validação funcional, organização do código e boa experiência de uso ao capturar dados do usuário.

A.	**FORMULÁRIOS COM REACT HOOK FORM** Avalia a implementação de formulários com React Hook Form, garantindo validação correta, captura eficiente de dados e código organizado.	**PONTUAÇÃO MÁXIMA** **5,0 pontos**
	REQUISITO	**PONTUAÇÃO**
a.	**USO OBRIGATÓRIO DO REACT HOOK FORM** Formulário com validação com useForm()	5,0 pontos

FRONT-END DESIGN ENGINEERING

CRITÉRIOS DE AVALIAÇÃO E PONTUAÇÃO (06/09)

6. VERSIONAMENTO DO PROJETO NO GITHUB

Avalia se o projeto foi versionado corretamente no GITHUB, incluindo organização de *commits*, uso adequado do repositório e manutenção do histórico de desenvolvimento.

A.	**CONTROLE COLABORATIVO DE CÓDIGO** Este critério avalia se o projeto foi versionado corretamente utilizando GIT e GITHUB, garantindo organização, rastreabilidade e colaboração entre os membros da equipe durante o desenvolvimento.	**PONTUAÇÃO MÁXIMA** **5,0 pontos**
	REQUISITO	**PONTUAÇÃO**
a.	**CRIAÇÃO DO REPOSITÓRIO** Criação de um repositório no GITHUB para armazenar e versionar o projeto.	1,5 ponto
b.	**LINK DO REPOSITÓRIO NO README.MD** Inclusão do link do repositório na seção indicada do README.md, permitindo acesso para correção e acompanhamento.	1,5 ponto
c.	**COMMITS FREQUENTES E RELEVANTES** Realização de **mínimo 5 commits por integrante**, mostrando evolução contínua do desenvolvimento.	1,0 ponto
d.	**PARTICIPAÇÃO COLABORATIVA DA EQUIPE** Contribuição visível de todos os membros no repositório, com histórico de *commits* individual.	1,0 ponto

FRONT-END DESIGN ENGINEERING

CRITÉRIOS DE AVALIAÇÃO E PONTUAÇÃO (07/09)

7. GUIA TÉCNICO E INFORMATIVO (1/2)

Avalia se o projeto foi versionado corretamente no GITHUB, incluindo organização de commits, uso adequado do repositório e manutenção do histórico de desenvolvimento.

A.	**DOCUMENTAÇÃO TÉCNICA DO PROJETO** Este critério avalia a qualidade do arquivo **README.md**, verificando se ele documenta o projeto de forma clara, completa e organizada, permitindo sua compreensão sem necessidade de analisar diretamente o código.	**PONTUAÇÃO MÁXIMA** **10,0 pontos**
	REQUISITO	**PONTUAÇÃO**
a.	**Título e Descrição** Apresentação do nome do projeto e breve resumo de sua finalidade	1,4 ponto
b.	**Tecnologias Utilizadas** Lista das ferramentas e linguagens utilizadas no desenvolvimento	1,4 ponto
c.	**Estrutura de Pastas do Projeto** Demonstração da organização interna do projeto	1,4 ponto
d.	**Autores e Créditos** Nome, RM, turma, foto e links do LINKEDIN e GITHUB de cada integrante	1,4 ponto
e.	**Imagens e Ícones do Projeto** Inserção de imagens ilustrativas e ícones relevantes do sistema	1,4 ponto

FRONT-END DESIGN ENGINEERING

CRITÉRIOS DE AVALIAÇÃO E PONTUAÇÃO (08/09)

7. GUIA TÉCNICO E INFORMATIVO (2/2)

Avalia se o projeto foi versionado corretamente no GITHUB, incluindo organização de commits, uso adequado do repositório e manutenção do histórico de desenvolvimento.

A.	**DOCUMENTAÇÃO TÉCNICA DO PROJETO** Este critério avalia a qualidade do arquivo **README.md**, verificando se ele documenta o projeto de forma clara, completa e organizada, permitindo sua compreensão sem necessidade de analisar diretamente o código.		**PONTUAÇÃO MÁXIMA** **10,0 pontos**
REQUISITO			**PONTUAÇÃO**
f.	**Como Usar** Instruções claras para instalação, execução e uso do projeto	*i.* *Link* do repositório do GITHUB	0,8 ponto
		ii. *Link* do vídeo no YOUTUBE	0,8 ponto
g.	**Contato** Informações para contato com a equipe		1,4 ponto

FRONT-END DESIGN ENGINEERING

CRITÉRIOS DE AVALIAÇÃO E PONTUAÇÃO (09/09)

8. VÍDEO

Avalia a apresentação em vídeo do projeto, verificando se demonstra claramente o funcionamento da aplicação e explica suas principais funcionalidades e tecnologias utilizadas.

A.	**DEMONSTRAÇÃO FUNCIONAL DO PROJETO** Este critério avalia a clareza e a qualidade da apresentação em vídeo, verificando se o grupo demonstra corretamente o funcionamento da aplicação, suas principais funcionalidades e tecnologias aplicadas.	**PONTUAÇÃO MÁXIMA** **5,0 pontos**
	REQUISITO	**PONTUAÇÃO**
a.	**Gravação de Vídeo** O grupo deve gravar um vídeo de apresentação do projeto com duração de **até 3 minutos**, mostrando telas, layout, navegação e principais recursos da aplicação.	2,5 pontos
b.	**Publicação no YouTube** O vídeo deve ser publicado no **YouTube** e o link deve ser disponibilizado no arquivo **README.md**.	2,5 pontos

FRONT-END DESIGN ENGINEERING

PENALIDADES (01/05)

As penalidades abaixo serão aplicadas ao projeto caso algum requisito obrigatório não seja atendido. Em casos graves, o projeto poderá ser **zerado**.

ENTREGA DO PROJETO	
INFRAÇÃO	**PENALIDADE**
Arquivo .ZIP maior que 50MB	–50,0 pontos
Entrega apenas com link do repositório (sem ZIP)	–100,0 pontos
Projeto sem histórico Git (sem .git no ZIP)	–50,0 pontos
Envio do projeto com a pasta node_modules	–20,0 pontos

FRONT-END DESIGN ENGINEERING

PENALIDADES (02/05)

As penalidades abaixo serão aplicadas ao projeto caso algum requisito obrigatório não seja atendido. Em casos graves, o projeto poderá ser **zerado**.

PÁGINAS E ESTRUTURA OBRIGATÓRIA	
INFRAÇÃO	**PENALIDADE**
Página obrigatória faltando (Home, Sobre, FAQ, Contato, Integrantes, Solução)	–20,0 pontos por página
Página de Integrantes ausente	–20,0 pontos
Item obrigatório faltando na página de Integrantes (Nome, RM, Turma, LinkedIn, GitHub)	–10,0 pontos por item
Páginas não convertidas para React (HTML puro)	–30,0 pontos por página

FRONT-END DESIGN ENGINEERING

PENALIDADES (03/05)

As penalidades abaixo serão aplicadas ao projeto caso algum requisito obrigatório não seja atendido. Em casos graves, o projeto poderá ser **zerado**.

PROIBIÇÕES TÉCNICAS (ZERAÇÃO)	
INFRAÇÃO	**PENALIDADE**
Uso de qualquer framework proibido (Bootstrap, Material UI, Chakra UI, jQuery etc.)	Nota Zerada
Uso de Axios ou bibliotecas externas não autorizadas	Nota Zerada
Uso de templates prontos da internet	Nota Zerada
Uso de qualquer outro framework que não seja React + Vite (como NextJS, Vue, Angular, CRA, React puro sem Vite)	Nota Zerada
Trabalho idêntico entre grupos (plágio)	Nota Zerada+ Abertura de ocorrência disciplinar

FRONT-END DESIGN ENGINEERING

PENALIDADES (04/05)

As penalidades abaixo serão aplicadas ao projeto caso algum requisito obrigatório não seja atendido. Em casos graves, o projeto poderá ser **zerado**.

GITHUB E VERSIONAMENTO	
INFRAÇÃO	**PENALIDADE**
Projeto sem link do GitHub no README.md	–50,0 pontos
Menos de 15 commits no total do projeto	–10,0 pontos
Menos de 5 commits por integrante	–10,0 pontos por integrante
Falta de participação de algum integrante	–15,0 pontos

FRONT-END DESIGN ENGINEERING
(21/22)

PENALIDADES (05/05)

As penalidades abaixo serão aplicadas ao projeto caso algum requisito obrigatório não seja atendido. Em casos graves, o projeto poderá ser **zerado**.

DOCUMENTAÇÃO	
INFRAÇÃO	**PENALIDADE**
README.md ausente	–50,0 pontos
README incompleto (sem descrição, links ou instruções)	–20,0 pontos
Vídeo de apresentação não enviado	–20,0 pontos por integrante
Vídeo com mais de 3 minutos ou sem explicação técnica	–10,0 pontos

FRONT-END DESIGN ENGINEERING

ENTREGA DO PROJETO

A equipe deverá enviar um único arquivo compactado (.ZIP) contendo:

- Arquivos do projeto, organizados corretamente, incluindo:
  - Páginas HTML
  - Estilos CSS
  - Pasta /assets com imagens, ícones e mídias (se houver)

- Arquivo de documentação **README.md**, obrigatoriamente contendo:
  - Todos os itens constantes no **ITEM 07**: **GUIA TÉCNICO E INFORMATIVO**

ONDE DEVE SER ENTREGUE

A entrega deve ser realizada **exclusivamente pelo Portal do Aluno**, na área da disciplina correspondente, anexando o arquivo **.ZIP** dentro do prazo estipulado.

NÃO SERÃO ACEITAS ENTREGAS POR E-MAIL,

MICROSOFT TEAMS OU QUALQUER OUTRO MEIO DE COMUNICAÇÃO!!

SOFTWARE ENGINEERING AND BUSINESS MODEL

Documento Funcional – Sprint 3 (20 pontos)

Este entregável tem como objetivo consolidar e corrigir os tópicos pendentes da Sprint 3, garantindo que cada equipe apresente um documento completo e funcional da solução em desenvolvimento. - Como uma proposta para o seu cliente – Turma do Bem

Estrutura obrigatória do documento

- **Capa**
  - Nome da empresa fictícia.
  - Nome da solução desenvolvida.
- **Sumário**
  - Organização do documento, indicando seções e subseções.
- **Descritivo da solução**
  - Explicação clara sobre o que é a solução.
  - Problema que resolve e contexto de aplicação.
- **Diferenciais em relação aos concorrentes**
  - Comparação com soluções similares existentes no mercado.
  - Destaque dos pontos fortes e inovações.

SOFTWARE ENGINEERING AND BUSINESS MODEL

- **Valor agregado ao público**
  - Benefícios diretos para os usuários.
  - Impacto esperado (eficiência, acessibilidade, inovação, etc.).
- **Business Model Canvas (BMC)**
  - Representação visual do modelo de negócio da solução.
- **Mapa de Empatia**
  - Identificação das necessidades, dores e expectativas do público-alvo.
- **Link da Gestão do projeto no Jira/Trello**
  - Registro atualizado de todas as entregas das disciplinas.
  - Continuidade da organização iniciada na Sprint 2.
- **Diagrama de Caso de uso**
  - Representação dos principais atores e suas interações com o sistema.
  - Deve evidenciar os fluxos de funcionalidades essenciais
- **Diagrama de Atividades**
  - Modelagem dos processos internos da solução.
  - Demonstração dos fluxos de trabalho e decisões.
- **Link do Pitch do protótipo de média fidelidade**
  - Apresentação do protótipo com base nas **10 Heurísticas de Nielsen** e nos **Fundamentos de UX Writing**.
  - Deve demonstrar clareza, usabilidade e foco na experiência do usuário.

SOFTWARE ENGINEERING AND BUSINESS MODEL

Diagrama de Casos de Uso - Instruções (20 pontos)

Cada equipe deverá elaborar diagramas completos de Casos de Uso que expliquem as funcionalidades principais do sistema.

- É obrigatório representar as funcionalidades da solução utilizando o ASTAH.
- Devem ser produzidos mínimo dois diagramas de caso de uso, cobrindo diferentes aspectos da solução.
- Cada diagrama deve ser acompanhado de um documento descritivo, explicando:
  - Os atores envolvidos.
  - Os casos de uso representados.
  - O fluxo básico e possíveis fluxos alternativos.
- O uso correto dos conceitos de UML é indispensável. Diagramas sem descrição ou feitos em ferramentas diferentes do ASTAH não serão aceitos.

SOFTWARE ENGINEERING AND BUSINESS MODEL

Diagrama de Atividades - Instruções (25 pontos)

Além dos casos de uso, cada equipe deverá elaborar diagramas de atividades que representem os processos internos da solução.

- O diagrama deve mostrar o fluxo de trabalho e as decisões que ocorrem dentro do sistema.
- É obrigatório utilizar o ASTAH para a modelagem.
- Cada diagrama deve ser acompanhado de um texto explicativo, descrevendo:
    - O objetivo do processo modelado.
    - As etapas representadas.
    - As condições de decisão e os resultados esperados.
- Diagramas sem documentação descritiva ou que não utilizem os conceitos corretos de UML não serão aceitos.

SOFTWARE ENGINEERING AND BUSINESS MODEL

Pitch do Protótipo de Média Fidelidade (35 pontos)

Cada equipe deverá preparar um vídeo de até 3 minutos apresentando o protótipo de média fidelidade.
Requisitos obrigatórios:

- Demonstrar como o protótipo aplica as 10 Heurísticas de Nielsen (ex.: feedback visual, consistência, prevenção de erros).
- Evidenciar os Fundamentos de UX Writing (clareza, consistência, tom de voz adequado, textos orientados à ação).
- Relacionar cada tela ou interação a pelo menos uma heurística e um princípio de UX Writing.

Formato:

- Vídeo curto e objetivo (máx. 3 minutos).
- Navegação prática pelo protótipo, sem leitura de slides.
- Explicação direta e demonstrativa.

- Cada equipe deverá entregar apenas um único arquivo contendo todos os itens solicitados; caso sejam enviados mais de um arquivo, haverá desconto de 10 pontos, pois a organização e a gestão da entrega são parte fundamental da avaliação desta disciplina.

Sprint 4

ARTIFICIAL INTELLIGENCE e CHATBOT

SPRINT 04 – Treino, validação e deploy do modelo preditivo (API com flask/streamlit)

A sprint 4 foca na parte prática de treinamento, validação, e disponibilização do modelo desenvolvido a partir dos dados validados na sprint 3. Leve em consideração o feedback e comentários feitos na sprint 3 à respeito da qualidade dos dados apresentados, e na sua coerência com o problema que pretendem resolver.

O objetivo nessa sprint é treinar um modelo preditivo robusto, por meio do treino e validação de diferentes modelos e escolha de um deles para colocar em produção, e então expor esse modelo como uma API que possa ser consumida por uma página web para predições em tempo real.

Objetivos

- Implementar uma pipeline de pré processamento dos dados;
- Treinar e validar diferentes modelos;
- Serializar o modelo;
- Realizar deploy como uma API usando python.

ARTIFICIAL INTELLIGENCE e CHATBOT

Entregáveis e pontuação

- (40 pontos): **Pipeline e seleção de modelo (.ipynb).** Faça uma pipeline de pré processamento que codifique categorias, tenha normalização (se necessário), e tenha seleção e engenharia de features. Os dados, após o pré processamento, devem ser divididos em uma partição de treino e testes. Sua pipeline também deve ter o treinamento de pelo menos 3 modelos diferentes (p.ex. Regressão linear, random forests, XGBoost, SVM, etc), selecionando um dos modelos como o melhor, baseado no desempenho na partição de testes.
- (40 pontos): **Serialização (.joblib) e deploy (.py)**. Serialize o modelo final utilizando o joblib, para que o serviço inteiro fique contido dentro de um único objeto que possa ser facilmente copiado para outras máquinas. Após isso, implemente um servidor que exponha ao menos os endpoints
  - /predict (recebe um JSON com os inputs do usuário, e retorna a predição como um JSON);
  - /health (endpoint simples para verificar se o serviço está ativo e operante).

ARTIFICIAL INTELLIGENCE e CHATBOT

Entregáveis e pontuação

- (20 pontos): **Documento com demonstrações (.pdf).** Crie um documento PDF que contenha capturas de tela da página web consumindo o serviço, mostrando a entrada do usuário e a predição sendo corretamente exibida. Também coloque instruções de como executar o serviço localmente, como um passo a passo detalhado para garantir que o deploy seja documentado e todo o processo seja reprodutível em outra máquina.

Uma dica para elaboração do passo a passo é solicitar para outro integrante do grupo que tente reproduzir todas as instruções em sua máquina. Um passo a passo que não funcione corretamente poderá implicar em penalidades na nota, pois a implementação apresentada deve ser facilmente portável para outras máquinas.

BUILDING RELATIONAL DATABASE

- Criando a base de dados (estrutura) e realizando a imputação de dados (inserts)

- Entregar a documentação da Sprint 3. - **10 pontos**

- Criar a estrutura atualizada da versão anterior para implementação de dados, ou seja, utilizar os comandos DDL.
- **10 pontos**

- Cada tabela da estrutura, deverá ser alimentada com 7 linhas de dados através de comandos DML – insert. - **10 pontos**

- Realizar a execução de comandos de atualização (update) e eliminação de dados (delete), aplicar 3 testes de atualização e eliminação de dados em tabelas a sua escolha. - **10 pontos, sendo 5 para 3 updates e 5 para 3 deletes**

- Atenção: uso da linguagem Oracle Sql

BUILDING RELATIONAL DATABASE

- Criando relatórios, Sql Oracle
- Relatório utilizando classificação de dados, a escolha da tabela é decisão do grupo.[**5 pontos**]
- Relatório utilizando alguma função do tipo numérica simples.[**5 pontos**]
- Relatório utilizando alguma função de grupo. [**15 pontos**]
- Relatório utilizando sub consulta. [**15 pontos**]
- Relatório utilizando junção de tabelas [**20 pontos**]
- Pelo menos 1 relatório de cada versão solicitada.
    - Formato de entrega, documento em pdf com a documentação solicitada anterirormente, arquivo sql ou txt com a implementação SQL ORACLE.

BUILDING RELATIONAL DATABASE

- Atenção as penalidades:

  - Entregar arquivo fora do padrão especificado na documentação > -25% da nota total
  - Não usar a linguagem SQL solicitada na documentação > - 50% da nota total
  - Usar ferramenta de modelagem diferente da solicitada na documentação > - 25% da nota total
  - Não idenficar os integrantes do grupo nos arquivos > - 30% da nota total
  - Não criar a codificação DDL inicial para dropar as tabelas > - 50% da nota total
  - Dados que não correspondam com a coluna utilizada > - 50% da nota total
  - Instrução com erro de execução será desconsiderada, nota = 0 (zero)
  - Relatório deverá obrigatoriamente retornar dados, caso não retorne será desconsiderado, nota = 0 (zero)

- Padrão a ser seguido no arquivo de codificação:

  - Primeiras linhas > ra e nome completo em ordem alfabética
  - Comentar o que a codificação a seguir irá realizar, por exemplo:
    - Apagando as tabelas....
    - Criando as tabelas.....
    - Inserindo dados na tabela.....
    - Criando relatório.....

COMPUTATIONAL THINKING USING PYTHON

Objetivo:
Tendo como referência o entregável da Sprint 3, a equipe deverá realizar a integração do sistema com o banco de dados ORACLE.

Entrega 1 (70 pontos) - Código Fonte (arquivos .py):

- O Código Fonte deve contemplar os seguintes requisitos:
  - [15 pontos] Implementar um CRUD, com pelo menos uma operação de inserção, alteração, exclusão e consulta;
  - [5 pontos] Estrutura de menus e submenus para acesso às opções do CRUD;
  - [10 pontos] Realizar validações em entrada de usuário e tratar exceções utilizando (utilizar try/except);
  - [15 pontos] Realizar ao menos duas consultas no banco de dados utilizando filtros (utilizar select/where) e disponibilizar uma opção para exportar os dados dessas consulta para um arquivo JSON;
  - [10 pontos] Consumo de uma API externa pública;
  - [5 pontos] Organizar o código fonte em funções;
  - [10 pontos] Garantir a usabilidade e a boa experiência do usuário e aplicar boas práticas de programação.

COMPUTATIONAL THINKING USING PYTHON

Entrega 2 (30 pontos) - Vídeo Explicativo:
Produzir um vídeo explicando como o programa aborda o problema e demonstrando o software em funcionamento, com duração máxima de 5 minutos.

- Destacar as funcionalidades implementadas e estruturas de programação utilizadas;
- O vídeo deve ser disponibilizado no Youtube (não esquecer de deixar o vídeo acessível).

Formato de Entrega: Um único arquivo em formato .ZIP contendo:

- Código fonte completo do sistema;
- Arquivo TXT com Nomes e RMs dos alunos do grupo, e o link para acesso ao vídeo disponibilizado no Youtube;
- Arquivo TXT com scripts necessários para criação das tabelas em banco de dados Oracle.

Penalidades:

- [70 pontos] - Soluções que não se adequam ao problema proposto (Não façam implementações genéricas como cadastro de usuários. Escolham funcionalidades específicas do projeto);
- [20 pontos] - CRUD incompleto;
- [30 pontos] - Vídeo não acessível ou link quebrado;
- [30 pontos] - Ausência dos scripts para criação das tabelas em banco de dados;
- [20 pontos] - Entrega fora dos padrões solicitados.

DOMAIN DRIVEN DESIGN USING JAVA (1 de 6)

Descrição:
Esta é a entrega final do projeto Java, que visa avaliar o desenvolvimento completo da aplicação, tanto em nível técnico quanto documental. O projeto deve conter todos os elementos necessários para o funcionamento do sistema de forma integrada com o front-end, além de uma documentação que comprove a estrutura e funcionamento da aplicação.

- A entrega deve ser feita por meio de um **repositório no GitHub**, contendo:
- O **código-fonte Java** atualizado.
- A **documentação final em formato PDF.**

Requisitos

Documentação 15 Pontos

A documentação deve estar clara, devidamente nomeada, organizada e conter obrigatoriamente os seguintes itens:

Capa

- Nome dos integrantes.
- Nome da equipe.
- Nome da solução.

DOMAIN DRIVEN DESIGN USING JAVA (2 de 6)

Objetivo e escopo do projeto

- Descrição objetiva e concisa da solução proposta.

Breve descrição das principais funcionalidades da solução

- Destaque das funcionalidades implementadas atualizadas.
- Pode ser em forma de lista ou tópicos com descrição.
- Explicação de cada método lógico, com o print de cada um dos 4 métodos exigidos.

Tabela de Endpoints (API Restful)

- URIs (caminhos completos dos recursos).
- Verbos HTTP (GET, POST, PUT, DELETE etc.).
- Códigos de status de resposta esperados (200, 404, 500 etc.).

Protótipo – Prints das telas implementadas

- Capturas de tela reais do sistema com explicações resumidas.

DOMAIN DRIVEN DESIGN USING JAVA (3 de 6)

Modelo de Entidade-Relacionamento (MER)

- Diagrama do banco de dados com entidades, atributos, chaves e relacionamentos.

Diagrama de Classes atualizado

- Representação das classes com atributos, métodos e relações (associação, herança etc.).

Obs. conter informações e procedimentos para rodar a aplicação conforme a solicitada na Sprint anterior anterior, porém atualizada.

Projeto Java finalizado (Código-Fonte) 85 Pontos

O sistema deve estar completo, funcional e estruturado conforme boas práticas:

Camada Modelo (entities, beans) [10 pontos]

- Classes modelo corretamente estruturadas, com atributos, construtores, métodos getters e setters.
- Conter no mínimo 6 classes, padrões de encapsulamento e boa prática. representando corretamente o diagrama de classes e o modelo de banco de dados.

DOMAIN DRIVEN DESIGN USING JAVA (4 de 6)

Conexão com o Banco de Dados e Métodos com lógica de negócio [30 pontos]

- Utilização de dependêcias no pom.xml, usuário de senhas para conexão.
- Implementação de no mínimo 4 métodos relevantes.
- Os métodos devem conter lógica e regras de negócio condizentes com a proposta do sistema.
- A complexidade e a criatividade da solução serão consideradas na avaliação.

Camadas Exceções, DAO e BO [10 pontos]

- Tratamento adequado de exceções.
- Contém todas as funcionalidades essenciais para o front-end.
- O CRUD (Create, Read, Update, Delete) deve estar **funcionando completamente**.
- Contém validações e regras de negócio adequadas.

API Restful [35 pontos]

- Implementação de todos os endpoints necessários para funcionamento do front-end através do Quarkus.
- Deve seguir os princípios REST (recursos, verbos HTTP, status de resposta, etc.).

DOMAIN DRIVEN DESIGN USING JAVA (5 de 6)

Penalidades:

Situação	Penalidade
Documentação desorganizada.	- 5 até 20 pontos.
Documentação em outro formato que não seja PDF.	-10 até 20 pontos.
Erro de nomenclatura no projeto.	-10 a -30 pontos (dependendo da gravidade).
Falta de organização no código.	-5 a -20 pontos.
Baixa resolução das imagens na documentação.	-5 pontos por item.
Sem link do GitHub	-35 a – 85 pontos.
Entrega após o prazo (não serão aceitos)	Penalização total (-100 pontos)

DOMAIN DRIVEN DESIGN USING JAVA (6 de 6)

Penalidades:

Situação	Penalidade
Ausência de qualquer item obrigatório na documentação.	- até 5 pontos por item, além da pontuação atribuída.
Entrega fora do padrão solicitado.	-10 pontos
Código que não compila ou não executa corretamente.	-20 a -50 pontos (dependendo da gravidade)
Falta de pelo menos um dos métodos com lógica de negócio.	-5 a -10 pontos por método faltante
CRUD incompleto ou com falhas graves.	-10 pontos por item
Entrega após o prazo (não serão aceitos).	Penalização total (-100 pontos)

FRONT-END DESIGN ENGINEERING

(01/20)

SPRINT 04 – INTEGRAÇÃO COM BACKEND E CONSUMO DE API

A **SPRINT 04** marca a etapa final de desenvolvimento do projeto, consolidando todo o trabalho realizado nas fases anteriores e evoluindo a solução para um nível profissional de entrega. Nesta etapa, a aplicação deverá ser concluída utilizando **React + Vite + TypeScript**, mantendo a arquitetura modular e o padrão **SPA (Single Page Application)** desenvolvidos na **SPRINT 03**.

Além da continuidade estrutural e visual, a **SPRINT 04** tem como principal objetivo **integrar funcionalidades reais ao projeto por meio de consumo de APIs**, conectando o **FRONT-END** com os serviços desenvolvidos na disciplina de ***Domain Driven Design Using Java***. Essa integração deve ser feita por meio de requisições HTTP aos *endpoints* criados pelo grupo, garantindo comunicação entre cliente (Front-End) e servidor (Back-End).

Tecnologias obrigatórias nesta etapa: REACT, VITE e **TYPESCRIPT**.

FRONT-END DESIGN ENGINEERING

(02/20)

OBJETIVO

Desenvolver uma aplicação Web completa utilizando **React + Vite + TypeScript**, aplicando boas práticas de **Front-End Design Engineering**, com foco em:

- **Componentização e arquitetura modular**, visando escalabilidade e facilidade de manutenção;
- **Integração com APIs REST**, consumindo endpoints desenvolvidos na disciplina de **Domain Driven Design Using Java**, garantindo comunicação consistente entre *frontend* e *backend*;
- **Abordagem Single Page Application (SPA)**, proporcionando navegação fluida e rápida sem recarregamento total da página;
- **Padrões de projeto modernos**, aplicando recursos do ecossistema React como hooks (useState, useEffect, useContext) e organização eficiente de componentes;
- **Tipagem estática com TypeScript**, garantindo maior confiabilidade e redução de erros durante o desenvolvimento;
- **Experiência do usuário coesa e responsiva**, com design acessível e alinhado às boas práticas de usabilidade;
- **Estilização com Tailwind CSS**, assegurando consistência visual e produtividade no desenvolvimento da interface;
- **Responsividade completa**, garantindo adaptação correta da interface para diferentes dispositivos e tamanhos de tela;
- **Qualidade técnica do código**, seguindo padrões de clean code e organização de pastas;
- **Versionamento colaborativo com GitHub**, registrando o histórico de desenvolvimento e garantindo integração contínua entre os membros da equipe;
- **Documentação técnica clara**, por meio de um arquivo README.md completo, que explique instalação, execução e estrutura do projeto;
- **Deploy profissional na Vercel**, disponibilizando a aplicação online para avaliação e acesso público.

FRONT-END DESIGN ENGINEERING

ALGUMAS REGRAS (01/04)

1. **FRAMEWORK**

   O projeto final deverá ser estruturado utilizando **REACT + VITE + TYPESCRIPT**, obrigatoriamente, onde o roteamento de páginas e outras adaptações necessárias serão implementados.

2. **RESPONSIVIDADE**

   A aplicação deve ser totalmente responsiva utilizando exclusivamente Tailwind CSS, garantindo funcionamento adequado em diferentes resoluções, incluindo Mobile (até 480px), Tablet (768px) e Desktop (992px ou superior), preservando a integridade do layout sem quebras de design em nenhuma dimensão de tela.

3. **DEPLOY DO PROJETO**

   Além disso é fundamental realizar o *deploy* do projeto na plataforma **VERCEL**. A URL aqui criada deve ser inserida no README.md, na seção "Como Usar", pois esta aplicação já deve estar consumindo remotamente a API criada na disciplina de Java para que os testes possam ser realizados.

FRONT-END DESIGN ENGINEERING

ALGUMAS REGRAS (02/04)

4. LINK DO GITHUB

A **EQUIPE** criar um repositório no GITHUB. A URL aqui criada deve ser inserida no README.md, na seção "Como Usar", para que a correção possa ser efetuada.

O versionamento deve estar presente no projeto. É obrigatório utilizar o **GIT/GITHUB/GITFLOW** para realizar o versionamento, todos os integrantes devem participar do trabalho, sendo possível ver esta participação através do histórico de *commits*, por isso o **mínimo de *commit* por integrante é 5** e o **mínimo total de *commits* do projeto é 15**.

5. GRAVAÇÃO DE VÍDEO

A equipe deve publicar um vídeo de apresentação do projeto no YouTube, de no máximo TRÊS minutos, demonstrando o funcionamento da aplicação, suas principais funcionalidades, tecnologias utilizadas e a integração com a API. A URL do vídeo deve ser inserida no arquivo README.md, na seção "Como Usar", para fins de avaliação.

5. APPLICATION PROGRAMMING INTERFACE (API) – JAVA – Obrigatória

A **API**, desenvolvida na disciplina de *Domain Drive Design Using Java*, será responsável por enviar e receber dados entre o *backend* e o *frontend*. Ela trará todos os dados coletados no *backend* para o *frontend*, possibilitando também o envio de dados do *frontend* para o *backend*, viabilizando seu armazenamento no banco de dados, conforme abordado na disciplina de *Building Relational Database*. ***Lembrando que tanto o front-end quanto o back-end devem estar publicados na nuvem.***

FRONT-END DESIGN ENGINEERING

ALGUMAS REGRAS (03/04)

7. PÁGINAS OBRIGATÓRIAS

Em seu projeto deve constar estas páginas além das páginas da aplicação — **index/home** (Página Inicial), **integrantes**, **sobre/about**, **FAQ** (Perguntas Frequentes) ou **contato**. Os nomes podem variar destes aqui apresentados, desde que representem as mesmas funções.

8. PÁGINAS DA SOLUÇÃO DO PROJETO

Além das páginas obrigatórias, o site deve conter mínimo de 2 páginas extras dedicadas à solução do problema proposto no desafio. Essas páginas devem:

- ✓ Apresentar de forma clara a solução desenvolvida pelo grupo.
- ✓ Conter conteúdo funcional ou prototipação navegável.
- ✓ Estar alinhadas com a proposta do projeto (tema e problema definidos).
- ✓ Ser integradas ao menu e à identidade visual do restante do site.

9. PÁGINA DE INTEGRANTES

A página de integrantes deve ser criada como uma página do sistema e deve constar dos links comuns com os seguintes dados:

- ✓ Nome, foto, RM, turma, além dos *links* do GITHUB e LINKEDIN de cada integrante

ALGUMAS REGRAS (04/04)

10. GUIA TÉCNICO E INFORMATIVO

O arquivo **README.md** é parte obrigatória da entrega e será utilizado como guia técnico do projeto no GitHub. Ele deve ser bem estruturado em Markdown e conter, no mínimo, as seguintes seções:

- ✓ Título e Descrição;
- ✓ Tecnologias utilizadas;
- ✓ Estrutura de Pastas;
- ✓ Autores e Créditos;
- ✓ Imagens e ícones relacionadas ao projeto;
- ✓ Como usar:
  - ✓ Esta seção deverá conter:
    - ✓ Link do repositório do GITHUB do projeto;
    - ✓ Link do vídeo do projeto, hospedado no YOUTUBE;
    - ✓ URL do projeto hospedado na plataforma Vercel
- ✓ e Contato.

FRONT-END DESIGN ENGINEERING

CRITÉRIOS DE AVALIAÇAO E PONTUAÇÃO (01/08)

1. CONSTRUÇÃO DO PROJETO (1/2)

A construção da solução para esta *Global Solution* deverá ser feita utilizando o *framework* **REACT + VITE + TYPESCRIPT.**

A. CRIAÇÃO DE ROTAS ESTÁTICAS E DINÂMICAS (PASSAGEM DE PARÂMETROS) COM VITE Este requisito avalia se o projeto implementa corretamente rotas estáticas e dinâmicas para navegação entre páginas, incluindo a passagem de parâmetros pelas rotas de forma funcional e organizada.		**PONTUAÇÃO MÁXIMA** **10,0 pontos**
REQUISITO	**SUB-REQUISITO**	**PONTUAÇÃO**
I. Criação de Rotas estáticas Implementação de rotas fixas no sistema, cujo caminho não muda, como /home, /contato ou /sobre.	**a. Definição clara e funcional de rotas** Estruturação correta das rotas com tipagem e organização.	2,5 pontos
	b. Navegação Navegação fluida e intuitiva entre as páginas, proporcionando uma experiência de usuário consistente	1,0 ponto
II. Criação de Rotas dinâmicas com passagem de parâmetros Implementação de rotas que recebem parâmetros variáveis na URL, permitindo exibir conteúdos diferentes com a mesma estrutura, como /produto/:id ou /usuario/:nome.	**a. Parâmetros de Rota Dinâmica** Tratamento correto de rotas dinâmicas com passagem de parâmetros.	5,0 pontos
	b. Redirecionamento Utilização eficaz de redirecionamentos e *feedbacks* ao usuários com mensagens personalizadas.	1,5 ponto

FRONT-END DESIGN ENGINEERING

CRITÉRIOS DE AVALIAÇAO E PONTUAÇÃO (02/08)

1. CONSTRUÇÃO DO PROJETO (2/2)

A construção da solução para esta *Global Solution* deverá ser feita utilizando o *framework* **REACT + VITE + TYPESCRIPT.**

B. UTILIZAÇÃO DE TIPOS ESPECÍFICOS (TYPESCRIPT) Este critério avalia se o projeto utiliza corretamente os tipos do TypeScript para garantir segurança e organização no código.		**PONTUAÇÃO MÁXIMA** **10,0 pontos**
REQUISITO	**SUB-REQUISITO**	**PONTUAÇÃO**
I. Utilização de tipos específicos (typescript) Criação e utilização correta de tipos.	**a. Criação de Tipos de Dados** Uso de tipos básicos como string, number, boolean e object.	4,0 pontos
	b. Tipos Avançados Uso de tipos avançados como **Union Types** e **Intersection Types**.	4,0 pontos
	c. Interface Uso correto de interface para tipagem personalizada de objetos.	2,0 ponto

FRONT-END DESIGN ENGINEERING

CRITÉRIOS DE AVALIAÇAO E PONTUAÇÃO (03/08)

2. ESTILIZAÇÃO E RESPONSIVIDADE COM TAILWIND

Este item avalia a qualidade visual da aplicação, verificando se o layout foi bem estilizado com Tailwind CSS e se a interface funciona corretamente em diferentes tamanhos de tela (responsividade).

A. ADAPTAÇÃO DO LAYOUT PARA DIFERENTES TELAS Este critério avalia se a aplicação está devidamente responsiva com Tailwind CSS, apresentando uma interface funcional e bem organizada em diferentes resoluções de tela: eXtra Small devices ; SMall devices; MeDium devices; Large Devices e eXtra-Large devices,		**PONTUAÇÃO MÁXIMA** **15,0 pontos**
REQUISITO	**SUB-REQUISITO**	**PONTUAÇÃO**
I. Adaptabilidade Avalia a capacidade de um sistema se ajustar a diferentes contextos ou condições mantendo sua funcionalidade.	**a. Funcionalidade e aparência adequada em diferentes dispositivos** Descreve **responsividade**, ou seja, a capacidade de um site manter boa funcionalidade e visual consistente em diferentes tamanhos de tela, como celular, tablet e desktop.	5,0 pontos
	b. Adequação do *layout* e conteúdo para diferentes tamanhos de tela / @media-queries Refere ao uso de **design responsivo**, aplicando técnicas como **media queries** para ajustar layout e conteúdo de acordo com diferentes resoluções de tela.	10,0 pontos

FRONT-END DESIGN ENGINEERING

(10/20)

CRITÉRIOS DE AVALIAÇAO E PONTUAÇÃO (04/08)

3. DEPLOY DO PROJETO PARA PLATAFORMA VERCEL

Avalia se o projeto foi publicado corretamente na plataforma Vercel e se o *link* de acesso está funcionando de forma pública e estável

A. PUBLICAÇÃO FUNCIONAL DO PROJETO Este critério avalia se o deploy na Vercel foi realizado com sucesso, garantindo que a aplicação esteja acessível online de forma pública, estável e sem erros.		**PONTUAÇÃO MÁXIMA** **10,0 pontos**
REQUISITO	**SUB-REQUISITO**	**PONTUAÇÃO**
I. Deploy Funcional e Acessível Garante que o projeto foi publicado corretamente na Vercel, está funcionando sem erros e pode ser acessado por qualquer usuário através de um link público.	**a. URL pública disponível** O link da aplicação hospedada na Vercel deve estar acessível e inserido no arquivo README.md.	5,0 pontos
	b. Deploy funcional O projeto deve estar publicado corretamente, rodando sem erros e disponível online na plataforma Vercel.	5,0 pontos

FRONT-END DESIGN ENGINEERING

CRITÉRIOS DE AVALIAÇAO E PONTUAÇÃO (05/08)

4. INTEGRAÇÃO ADEQUADA DAS APIs

Este critério avalia se a aplicação realiza corretamente a integração com a API desenvolvida em **Java (DDD)**, garantindo comunicação estável e eficiente com os dados.

A. INTEGRAÇÃO ENTRE FRONT-END E BACK-END Este critério avalia se as APIs foram corretamente conectadas ao projeto, garantindo comunicação estável com o back-end em Java (DDD) e funcionamento adequado das funcionalidades que dependem de dados externos.		**PONTUAÇÃO MÁXIMA** **10,0 pontos**
REQUISITO	**SUB-REQUISITO**	**PONTUAÇÃO**
I. Requisitos de Integração Os requisitos de integração avaliam como o front-end se comunica com a API, verificando se os dados são consumidos corretamente, se os métodos HTTP estão bem aplicados e se há tratamento adequado de respostas e erros.	**a. Consumo da API** A aplicação deve consumir os endpoints corretamente utilizando o padrão request/response.	10,0 pontos
	b. Manipulação dos verbos HTTP Deve implementar operações da API com os métodos HTTP adequados (**GET, POST, PUT, DELETE**) seguindo o modelo **CRUD**.	10,0 pontos
	c. Tratamento de dados e erros Deve manipular corretamente as respostas da API e implementar tratamento de erros e respostas inesperadas.	10,0 pontos

FRONT-END DESIGN ENGINEERING

CRITÉRIOS DE AVALIAÇAO E PONTUAÇÃO (06/08)

5. VERSIONAMENTO DO PROJETO NO GITHUB

Avalia se o projeto foi versionado corretamente no GITHUB, incluindo organização de *commits*, uso adequado do repositório e manutenção do histórico de desenvolvimento.

A. CONTROLE E ORGANIZAÇÃO DO REPOSITÓRIO Este critério avalia se o projeto está devidamente versionado no GITHUB, com *commits* bem estruturados, histórico claro de desenvolvimento e colaboração organizada entre os membros da equipe.		**PONTUAÇÃO MÁXIMA** **10,0 pontos**
REQUISITO	**SUB-REQUISITO**	**PONTUAÇÃO**
I. Requisitos de Versionamento Verificam se o projeto foi controlado corretamente no GITHUB, com repositório organizado, *commits* frequentes e participação colaborativa de todos os integrantes.	**a. Criação do Repositório** Criação de um repositório no GITHUB para armazenar e versionar o projeto.	2,5 pontos
	b. Link do Repositório no README.md Inclusão do link do repositório na seção indicada do README.md, permitindo acesso para correção e acompanhamento.	2,5 pontos
	c. Commits Frequentes e Relevantes Realização de **mínimo 5 commits por integrante**, mostrando evolução contínua do desenvolvimento.	2,5 pontos
	d. Participação Colaborativa da Equipe Contribuição visível de todos os membros no repositório, com histórico de *commits* individual.	2,5 pontos

FRONT-END DESIGN ENGINEERING

CRITÉRIOS DE AVALIAÇAO E PONTUAÇÃO (07/08)

6. GUIA TÉCNICO E INFORMATIVO

Avalia se o projeto foi versionado corretamente no GITHUB, incluindo organização de *commits*, uso adequado do repositório e manutenção do histórico de desenvolvimento.

A.	**DOCUMENTAÇÃO TÉCNICA DO PROJETO** Este critério avalia a qualidade do arquivo **README.md**, verificando se ele documenta o projeto de forma clara, completa e organizada, permitindo sua compreensão sem necessidade de analisar diretamente o código.	**PONTUAÇÃO MÁXIMA** **10,0 pontos**

REQUISITO	SUB-REQUISITO		PONTUAÇÃO
I. **Requisitos do Guia Técnico** Os requisitos do Guia Técnico garantem que o projeto esteja bem documentado no README.md, com informações claras sobre objetivo, estrutura, tecnologias, autoria e instruções de uso.	**a.** **Título e Descrição** Apresentação do nome do projeto e breve resumo de sua finalidade.		1,4 ponto
	b. **Tecnologias Utilizadas** Lista das ferramentas e linguagens utilizadas no desenvolvimento.		1,4 ponto
	c. **Estrutura de Pastas do Projeto** Demonstração da organização interna do projeto.		1,4 ponto
	d. **Autores e Créditos** Nome, RM, turma, foto e links do LINKEDIN e GITHUB de cada integrante.		1,4 ponto
	e. **Imagens e Ícones do Projeto** Inserção de imagens ilustrativas e ícones relevantes do sistema.		1,4 ponto
	f. **Como Usar** Instruções claras para instalação, execução e uso do projeto.	i. Link do repositório do GitHub	0,6 ponto
		ii. Link do vídeo no YouTube	0,5 ponto
		iii. URL do deploy na Vercel	0,5 ponto
	g. **Contato** Informações para contato com a equipe.		1,4 ponto

FRONT-END DESIGN ENGINEERING

CRITÉRIOS DE AVALIAÇAO E PONTUAÇÃO (08/08)

7. VÍDEO

Avalia a apresentação em vídeo do projeto, verificando se demonstra claramente o funcionamento da aplicação e explica suas principais funcionalidades e tecnologias utilizadas.

A.	**DEMONSTRAÇÃO FUNCIONAL DO PROJETO** Este critério avalia a clareza e a qualidade da apresentação em vídeo, verificando se o grupo demonstra corretamente o funcionamento da aplicação, suas principais funcionalidades e tecnologias aplicadas.		**PONTUAÇÃO MÁXIMA** **5,0 pontos**
	REQUISITO	**SUB-REQUISITO**	**PONTUAÇÃO**
I.	**Gravação de Vídeo** O grupo deve gravar um vídeo de apresentação do projeto com duração de **até 3 minutos**, mostrando telas, layout, navegação e principais recursos da aplicação.	**a.** **Publicação no YouTube** O vídeo deve ser publicado no **YouTube** e o link deve ser disponibilizado no arquivo **README.md**.	5,0 pontos

FRONT-END DESIGN ENGINEERING

(15/20)

PENALIDADES (01/05)

As penalidades abaixo serão aplicadas ao projeto caso algum requisito obrigatório não seja atendido. Em casos graves, o projeto poderá ser **zerado**.

ENTREGA DO PROJETO	
INFRAÇÃO	**PENALIDADE**
Arquivo .ZIP maior que 50MB	–50,0 pontos
Entrega apenas com link do repositório (sem ZIP)	–100,0 pontos
Projeto sem histórico Git (sem .git no ZIP)	–50,0 pontos
Envio do projeto com a pasta node_modules	–20,0 pontos

FRONT-END DESIGN ENGINEERING

(16/20)

PENALIDADES (02/05)

As penalidades abaixo serão aplicadas ao projeto caso algum requisito obrigatório não seja atendido. Em casos graves, o projeto poderá ser **zerado**.

PÁGINAS E ESTRUTURA OBRIGATÓRIA	
INFRAÇÃO	**PENALIDADE**
Página obrigatória faltando (Home, Sobre, FAQ, Contato, Integrantes, Solução)	–20,0 pontos por página
Página de Integrantes ausente	–20,0 pontos
Item obrigatório faltando na página de Integrantes (Nome, RM, Turma, LinkedIn, GitHub)	–10,0 pontos por item
Páginas não convertidas para React (HTML puro)	–30,0 pontos por página

FRONT-END DESIGN ENGINEERING

(17/20)

PENALIDADES (03/05)

As penalidades abaixo serão aplicadas ao projeto caso algum requisito obrigatório não seja atendido. Em casos graves, o projeto poderá ser **zerado**.

PROIBIÇÕES TÉCNICAS (ZERAÇÃO)	
INFRAÇÃO	**PENALIDADE**
Uso de qualquer framework proibido (Bootstrap, Material UI, Chakra UI, jQuery etc.)	Nota Zerada
Uso de Axios ou bibliotecas externas não autorizadas	Nota Zerada
Uso de templates prontos da internet	Nota Zerada
Uso de qualquer outro framework que não seja React + Vite (como NextJS, Vue, Angular, CRA, React puro sem Vite)	Nota Zerada
Trabalho idêntico entre grupos (plágio)	Nota Zerada+ Abertura de ocorrência disciplinar

FRONT-END DESIGN ENGINEERING

(18/20)

PENALIDADES (04/05)

As penalidades abaixo serão aplicadas ao projeto caso algum requisito obrigatório não seja atendido. Em casos graves, o projeto poderá ser **zerado**.

GITHUB E VERSIONAMENTO	
INFRAÇÃO	**PENALIDADE**
Projeto sem link do GitHub no README.md	–50,0 pontos
Menos de 15 commits no total do projeto	–10,0 pontos
Menos de 5 commits por integrante	–10,0 pontos por integrante
Falta de participação de algum integrante	–15,0 pontos

FRONT-END DESIGN ENGINEERING

(19/20)

PENALIDADES (05/05)

As penalidades abaixo serão aplicadas ao projeto caso algum requisito obrigatório não seja atendido. Em casos graves, o projeto poderá ser **zerado**.

DOCUMENTAÇÃO	
INFRAÇÃO	**PENALIDADE**
README.md ausente	–50,0 pontos
README incompleto (sem descrição, links ou instruções)	–20,0 pontos
Vídeo de apresentação não enviado	–20,0 pontos por integrante
Vídeo com mais de 3 minutos ou sem explicação técnica	–10,0 pontos

FRONT-END DESIGN ENGINEERING

(20/20)

ENTREGA DO PROJETO

A equipe deverá enviar um único arquivo compactado (.ZIP) contendo:

- Arquivos do projeto, organizados corretamente, incluindo:
  - Páginas HTML
  - Estilos CSS
  - Pasta /assets com imagens, ícones e mídias (se houver)

- Arquivo de documentação **README.md**, obrigatoriamente contendo:
  - Todos os itens constantes no **ITEM 06**: **GUIA TÉCNICO E INFORMATIVO**

ONDE DEVE SER ENTREGUE

A entrega deve ser realizada **exclusivamente pelo Portal do Aluno**, na área da disciplina correspondente, anexando o arquivo **.ZIP** dentro do prazo estipulado.

NÃO SERÃO ACEITAS ENTREGAS POR E-MAIL,

MICROSOFT TEAMS OU QUALQUER OUTRO MEIO DE COMUNICAÇÃO!!

SOFTWARE ENGINEERING AND BUSINESS MODEL

Documento Funcional

Cada grupo deverá entregar um único arquivo em PDF, padronizado com identidade visual (fontes e cores), contendo:

- Capa com o nome da startup.
- Nome completo e RM dos integrantes.
- Sumário e Sumário Executivo.

[10 pontos] Sobre o Negócio

- Detalhamento da oportunidade identificada.
- Modelo de negócios da startup.
- Modelo de relacionamento aplicado (B2B, B2C, etc.).
- Tipo de inovação da solução proposta.
- Business Model Canvas (BMC) atualizado.

[20 pontos] Mercado e Competidores

- Definição do mercado-alvo (segmento de clientes).
- Análise de concorrentes.
- Vantagem competitiva e diferenciais da solução proposta.

SOFTWARE ENGINEERING AND BUSINESS MODEL

[25 pontos] Precificação da Solução
- Racional das despesas para formação da hora técnica.
- Cálculo do valor da Hora/Homem.
- Preço final para apresentação ao cliente.

[25 pontos] Retorno do Investimento (ROI)
- Métricas tangíveis utilizadas.
- Racional do cálculo do ROI.

[20 pontos] Acordo de Nível de Serviço (SLA)
- Determinação dos serviços a serem prestados.
- Definição dos níveis de serviço.
- Disponibilidade dos serviços (datas, horários, etc.).
- Requisitos de Help Desk ou Service Desk.
- Informativo sobre multas/descontos em caso de descumprimento do SLA.

A entrega deve ser feita em um único arquivo consolidado (PDF). Caso sejam enviados múltiplos arquivos, haverá desconto de 10 pontos, pois a organização e a gestão da entrega são parte fundamental da avaliação.